

CSE215L: Programming Language Lab II

ASSIGNMENT

General Instructions:

1. Go through the [problem](#), [requirement analysis](#), [UML diagrams](#) and [class descriptions](#) carefully.
2. The solution that you submit must be your own work. You are expected to write the required methods by yourself, that is, without copying anyone else's coded solution. **Plagiarism** will lead you to a straight zero.
3. You have to follow the provided architecture to solve this problem.
4. Use proper **naming conventions** for naming fields. Most of the IDE's and code editors do proper indentation automatically. So, please use it.

Problem:

Suppose you are hired by a **Vinyl & CD shop** to develop an **Inventory Management and Billing** software for them. They have categorized their products into three main segments: **movies**, **games** and **music**. As a startup they are also offering different **discounts** on their products to attract customers **and may vary from product to product**, and they are calling it **product discount**. They have a special discount named **full house** which is if **someone buys at least two products from each category**, he/she will get a discount of **50% on the total price**. When billing an **invoice** if both of the discounts are eligible, the company wants to compare them. The discount that is larger of the two will be applied. **The company wants you to keep a functionality which will show them the inventory**. They also want to be able to **sort** their inventory according to: **price** and **available stocks**.

Requirement Analysis:

- ☐ **Product:** In the problem, the company has three types of products, which are **Music albums**, **Movies**, and **Games**. So we can see at the top of the hierarchy, all of them are "**Product**". Thus, we have a class named **Product**. The **Product** class has some data fields. Please take a look at the uml diagram. [learn more](#).
- ☐ **Stockable Product:** We have to stock each product, that means for each we can have many instances of a product. So, what we are doing here is, creating a class named **StockableProduct** which is keeping count of the number of instances we have stocked of a product by using a data field named **numberOfItemsStocked**. This is a specialized class of the **Product** class, which means this class has all the abilities of the **Product** class and on top of that this class can do some other operations which the **Product** class cannot. [Learn more](#).
- ☐ **Game:** As we have a type of product named **Game**, we have created a class called "**Game**" which is a specialized class of the **StockableProduct** class.

- ❑ **Music:** As we have a type of product named **Music**, we have created a class called “**Music**” which is a specialized class of the **StockableProduct** class.
- ❑ **Movie:** As we have a type of product named **Movie**, we have created a class called “**Movie**” which is a specialized class of the **StockableProduct** class.
- ❑ **Inventory:** Because this is an inventory management and billing system, we must have a class named **Inventory**. This class will basically hold the **StockableProducts**. In this case, we are taking a category based management system. Here we are going to think of each stockable product as a type / category of product. So we are going to store the stockable products in the inventory. And when someone wants a product from the inventory, we are going to give them an instance of the product. [Learn more.](#)

Example: Suppose, a shop has 20 Doritos. So in this case, Doritos is a product, and it has 20 instances. We can consider Doritos as a **StockableProduct**, and the number of items stocked is 20.

- ❑ **Invoice:** We also have to manage billing, so for billing we have to make a list of the products the buyer wants to buy. For this purpose, we are using a class named **Invoice**. When building the **Invoice** class, we are taking an item based approach, rather than taking a categorized approach as the Inventory. So, here we will take instance of a category, as a product. [Learn more.](#)

Example: If you think of the previous example, for the **Invoice** class, we are going to add 1 Doritos to the invoice from the 20 the shop had.

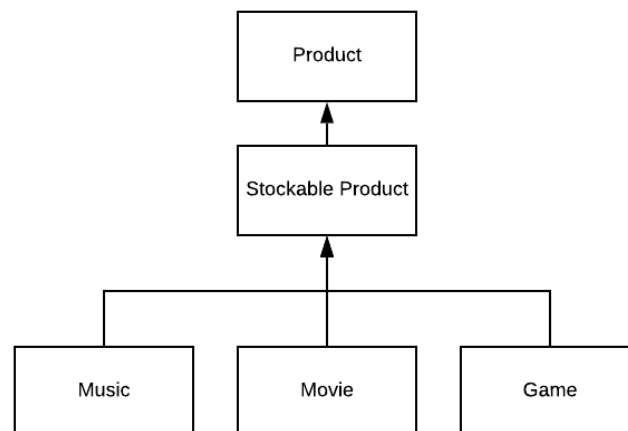


Figure 01: Hierarchical diagram of the products branch

UML Class Diagrams:

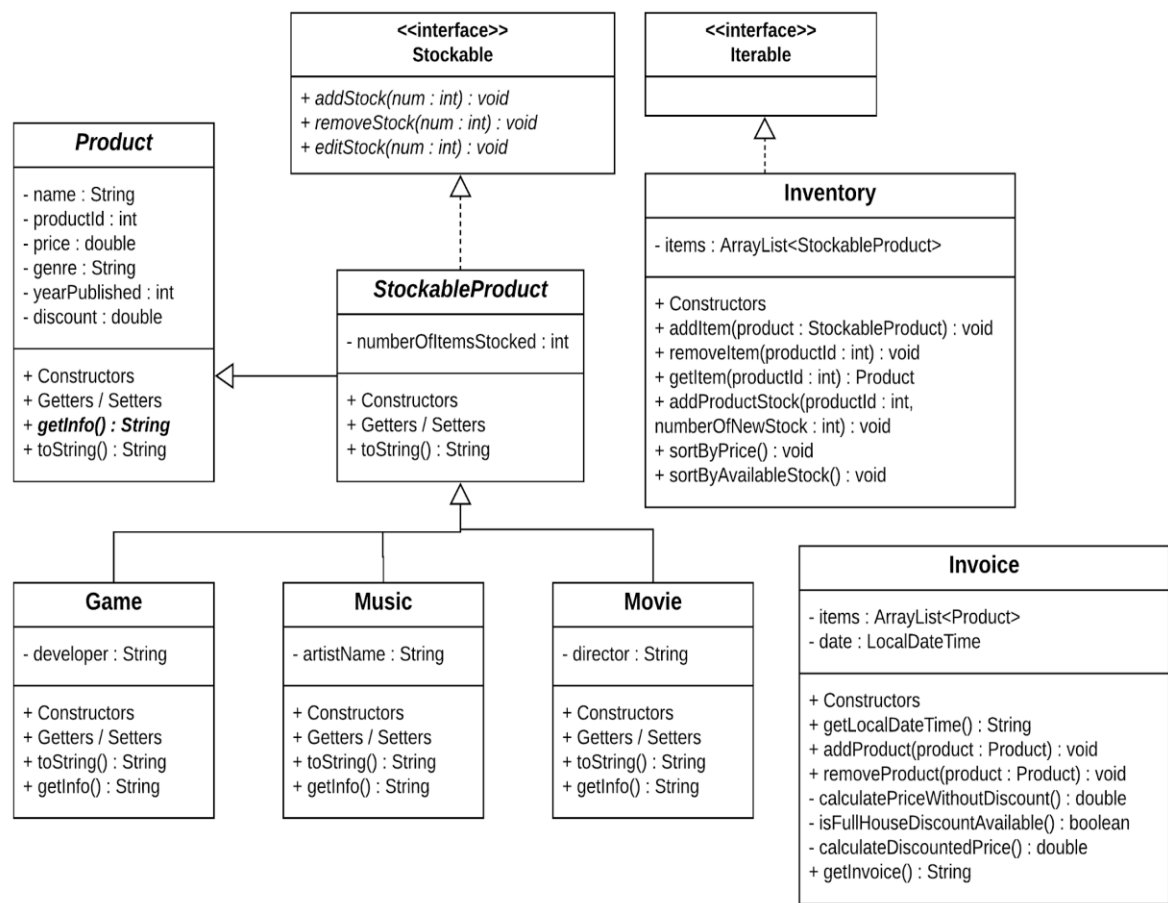


Figure 02: UML class diagrams

Class, Method & Variable Description:

Class: Product	
This class is an abstract class.	
<code>getInfo():String</code>	This method is an abstract method, and you will implement this method in the child classes(Game, Music, Movies).

Interface: Stockable	
This is an interface, by implementing this, you can make any class Stockable .	
addStock(int):void	This method will take the number of a product to be stocked, and add it with the number of the previously stocked product.
removeStock(int):void	This method will take the number of a product to be removed from the stock, and subtract it from the number of previously stocked product.
editStock(int):void	This method will take the new number of a product to be stocked and delete the previous number.

Class: StockableProduct
This class extends the class Product and implements the Stockable interface. Implement the methods of Stockable interface in this class.

Interface: Iterable
This is a default interface of java. By implementing this interface, you can make your collection of objects to be iterable by the foreach loop.

Class: Inventory	
This class is a wrapper of a data structure. In a data structure we do not have any functionality to print it from the object itself. So please do not write any method like displayInventory / printInventory, and print everything from there.	
addItem(StockableProduct):void	This method will take an object of StockableProduct as parameter and add that object to the items list.
removeItem(int):void	This method will take productId as parameter. Search for the object by product id and then remove that object from the items list.
getItem(int):Product	This method will take productId as parameter, and returns a Product from the stock of that StockableProduct .

	Hint: you have to decrement the numberOfItemsStocked , which is the count of stock of that specific product. You can use the removeStock method to do so.
addProductStock(int, int):void	This method will take productId and number of items to be stocked of a product as parameter. You have to search for that product, then add the number of items to be stocked to the numberOfItemsStocked field.
sortByPrice():void	This method will sort the Inventory by price of products. Note: This method will not print or return anything. So please do not print the items list from here. Hint: You can use the Comparator Interface of Java.
sortByAvailableStock():void	This method will sort the Inventory by available stock of products. Note: This method will not print or return anything. So please do not print the items list from here. Hint: You can use the Comparator Interface of Java.

Class: Invoice	
getLocalDateTime() : String	This method will return a String containing the formatted date and time. Ex: 2020-02-22 09:57:52
addProduct (Product) : void	This method will take an object of Product as parameter and add that product to the invoice. After adding a product to the invoice, you must reduce the numberOfItemsStocked of that product by 1, each time you add that product to an invoice.
removeProduct (Product) : void	This method will remove a specific product from the invoice and numberOfItemsStocked of that product will be incremented by 1 as the product gets back to inventory again.
calculatePriceWithOutDiscount():double	This method will calculate the total price without any discounts, and returns that sum.
isFullHouseDiscountAvailable():boolean	This method will check whether the fullHouse discount is available or not, and returns a boolean value depending on that. Build an algorithm for this

	method by yourself. Hint: There is a method named getClass(), play around with this. Convert class names into a set, {0, Music} {1, Movies} {2, Game}. Then you can map it into an array, and keep count. Indexes of array means Music, Movies or Games, and you can store the count in that.
calculateDiscounted price():double	In this method, you must calculate the total discounted price for regular discount, check whether fullHouse discount is available or not. If that is available then calculate the price after availing fullHouse discount and then compare between the two prices and return the one which is less.
getInvoice():String	This method will return a String which has a format like this, <pre>Date - yyyy-MM-dd HH:mm:ss Name: , Price: Name: , Price: Total Price: Price after discount:</pre>

Tasks:

Sl.	Tasks	Expected output
1.	Make an object of the Inventory class, Add all the input objects, and print information about the objects in the inventory. Hint: Because of implementing the Iterable interface now you can loop through the data structure from driver class, and you have the getInfo() method.	You know what the output will be.
2.	Create an invoice, and add m1,m2,v1,v2,g1 to the invoice object. Print the invoice. Hint: getInvoice()	<pre>2020-02-22 09:57:52 Name: submarine, Price: 20.0 Name: AM, Price: 10.0 Name: Good Will Hunting, Price: 12.0 Name: Life Is Beautiful, Price: 20.0 Name: Red Dead Redemption 2, Price: 29.0</pre>

		Total price: 91.0 Price after discount: 86.21
3.	Create an invoice, add m1,m2,v1,v2,g1,g2, and do same as above	2020-02-22 10:01:26 Name: submarine, Price: 20.0 Name: AM, Price: 10.0 Name: Good Will Hunting, Price: 12.0 Name: Life Is Beautiful, Price: 20.0 Name: Red Dead Redemption 2, Price: 29.0 Name: God of War, Price: 12.0 Total price: 103.0 Price after discount: 51.5
4.	Find available games, in the inventory object.	Red Dead Redemption 2 God of War Grand Theft Auto V
5.	Find the cheapest available Music Album, print name and price of the album	Name: AM, Price: 10.0
6.	Find All movies directed by "Francis Ford Coppola" Hint: you have the getInfo() method, use string split and string contains operations to extract director name from there	The Rainmaker The Godfather Apocalypse Now
7.	Show the currently available numberOfItemsStocked of each sold product.	Available Stocks of Sold Products in Inventory: Name: Submarine, Category: Music, Items Available: 98 Name: AM, Category: Music, Items Available: 8 Name: Good Will Hunting, Category: Movie, Items Available: 121 Name: Life Is Beautiful, Category: Movie, Items Available: 124 Name: Red Dead Redemption 2, Category: Game, Items Available: 558 Name: God of War, Category: Game, Items Available: 566